

# Threads

# Semaphores

CS232 Operating System, Fall 2019

# Introduction

- Semaphores invented by Dijkstra
- Can be used as locks as well as condition vars
- Two operations are defined:
  - `sem_wait()`
  - `sem_post()`

# Semaphores – operations

```
1  int sem_wait(sem_t *s) {  
2      decrement the value of semaphore s by one  
3      wait if value of semaphore s is negative  
4  }  
5  
6  int sem_post(sem_t *s) {  
7      increment the value of semaphore s by one  
8      if there are one or more threads waiting, wake one  
9  }
```

```
1  #include <semaphore.h>  
2  sem_t s;  
3  sem_init(&s, 0, 1);
```

# Semaphore – accessing critical section

```
1  sem_t m;  
2  sem_init(&m, 0, X); // initialize semaphore to X;  
3  
4  sem_wait(&m);  
5  // critical section here  
6  sem_post(&m);
```

- Initial value of X ??
- with  $X=1$ , this semaphore will behave as a lock!

# Semaphore – lock example

| Value | Thread 0                    | State   | Thread 1           | State    |
|-------|-----------------------------|---------|--------------------|----------|
| 1     |                             | Running |                    | Ready    |
| 1     | call sem_wait()             | Running |                    | Ready    |
| 0     | sem_wait() returns          | Running |                    | Ready    |
| 0     | (crit sect: begin)          | Running |                    | Ready    |
| 0     | <i>Interrupt; Switch→T1</i> | Ready   |                    | Running  |
| 0     |                             | Ready   | call sem_wait()    | Running  |
| -1    |                             | Ready   | decrement sem      | Running  |
| -1    |                             | Ready   | (sem<0)→sleep      | Sleeping |
| -1    |                             | Running | <i>Switch→T0</i>   | Sleeping |
| -1    | (crit sect: end)            | Running |                    | Sleeping |
| -1    | call sem_post()             | Running |                    | Sleeping |
| 0     | increment sem               | Running |                    | Sleeping |
| 0     | wake(T1)                    | Running |                    | Ready    |
| 0     | sem_post() returns          | Running |                    | Ready    |
| 0     | <i>Interrupt; Switch→T1</i> | Ready   |                    | Running  |
| 0     |                             | Ready   | sem_wait() returns | Running  |
| 0     |                             | Ready   | (crit sect)        | Running  |
| 0     |                             | Ready   | call sem_post()    | Running  |
| 1     |                             | Ready   | sem_post() returns | Running  |

- This is called a Binary Semaphore

# Semaphores – for ordering

```
1  sem_t s;
2
3  void *child(void *arg) {
4      printf("child\n");
5      sem_post(&s); // signal here: child is done
6      return NULL;
7  }
8
9  int main(int argc, char *argv[]) {
10     sem_init(&s, 0, X); // what should X be?
11     printf("parent: begin\n");
12     pthread_t c;
13     Pthread_create(&c, NULL, child, NULL);
14     sem_wait(&s); // wait here for child
15     printf("parent: end\n");
16     return 0;
17 }
```

- As a CV here
- Initial value??

# Semaphore – Producer-consumer problem

```
1  int buffer[MAX];
2  int fill = 0;
3  int use  = 0;
4
5  void put(int value) {
6      buffer[fill] = value;
7      fill = (fill + 1) % MAX;
8  }
9
10 int get() {
11     int tmp = buffer[use];
12     use = (use + 1) % MAX;
13     return tmp;
14 }
15
16 int main(int argc, char *argv[])
17     // ...
18     sem_init(&empty, 0, MAX);
19     sem_init(&full, 0, 0);
20     // ...
21 }
```

```
1  sem_t empty;
2  sem_t full;
3
4  void *producer(void *arg) {
5      int i;
6      for (i = 0; i < loops; i++) {
7          sem_wait(&empty);
8          put(i);
9          sem_post(&full);
10     }
11 }
12
13 void *consumer(void *arg) {
14     int i, tmp = 0;
15     while (tmp != -1) {
16         sem_wait(&full);
17         tmp = get();
18         sem_post(&empty);
19         printf("%d\n", tmp);
20     }
21 }
```

// Line P1  
// Line P2  
// Line P3  
  
// Line C1  
// Line C2  
// Line C3

# Semaphores – Producer Consumer w/ locks

```
1  sem_t empty;
2  sem_t full;
3  sem_t mutex;
4
5  void *producer(void *arg) {
6      int i;
7      for (i = 0; i < loops; i++)
8          sem_wait(&mutex);
9          sem_wait(&empty);
10         put(i);
11         sem_post(&full);
12         sem_post(&mutex);
13     }
14 }
15
16 void *consumer(void *arg) {
17     int i;
18     for (i = 0; i < loops; i++)
19         sem_wait(&mutex);
20         sem_wait(&full);
21         int tmp = get();
22         sem_post(&empty);
23         sem_post(&mutex);
24         printf("%d\n", tmp);
25     }
26 }
```

```
int main(int argc, char *argv[]) {
    // ...
    sem_init(&empty, 0, MAX); // MAX are empty
    sem_init(&full, 0, 0);    // 0 are full
    sem_init(&mutex, 0, 1);   // mutex=1: a lock
    // ...
}
```

# Semaphore – Producer consumer (correct)

```
1  sem_t empty;
2  sem_t full;
3  sem_t mutex;
4
5  void *producer(void *arg) {
6      int i;
7      for (i = 0; i < loops; i++) {
8          sem_wait(&empty);          /
9          sem_wait(&mutex);          /
10         put(i);                    /
11         sem_post(&mutex);           /
12         sem_post(&full);            /
13     }
14 }
15
16 void *consumer(void *arg) {
17     int i;
18     for (i = 0; i < loops; i++) {
19         sem_wait(&full);            /
20         sem_wait(&mutex);           /
21         int tmp = get();            /
22         sem_post(&mutex);           /
23         sem_post(&empty);           /
24         printf("%d\n", tmp);
25     }
26 }
```

```
int main(int argc, char *argv[]) {
    // ...
    sem_init(&empty, 0, MAX); // MAX are empty
    sem_init(&full, 0, 0);    // 0 are full
    sem_init(&mutex, 0, 1);   // mutex=1: a lock
    // ...
}
```

# Reader-Writer locks

- Let's say a concurrent operations happen on a global linked-list
- Of two kinds:
  - Write(), modify the list
  - Read(), simply read list data
- Multiple threads
- Can we have multiple reads in parallel? How?

```

typedef struct _rwlock_t {
    sem_t lock;          // binary semaphore
    sem_t writelock;     // allow ONE writer
    int    readers;      // #readers in critical section
} rwlock_t;

void rwlock_init(rwlock_t *rw) {
    rw->readers = 0;
    sem_init(&rw->lock, 0, 1);
    sem_init(&rw->writelock, 0, 1);
}

void rwlock_acquire_readlock(rwlock_t *rw) {
    sem_wait(&rw->lock);
    rw->readers++;
    if (rw->readers == 1) // first reader gets writelock
        sem_wait(&rw->writelock);
    sem_post(&rw->lock);
}

void rwlock_release_readlock(rwlock_t *rw) {
    sem_wait(&rw->lock);
    rw->readers--;
    if (rw->readers == 0) // last reader lets it go
        sem_post(&rw->writelock);
    sem_post(&rw->lock);
}

void rwlock_acquire_writelock(rwlock_t *rw) {
    sem_wait(&rw->writelock);
}

void rwlock_release_writelock(rwlock_t *rw) {
    sem_post(&rw->writelock);
}

```

# Dining Philosophers Problem

- N philosophers sitting on a table
- N forks b/w them. One b/w each pair.
- A philosopher can do two things:
  - Philosophize ( no need of forks)
  - Eat (needs both forks: left and right)

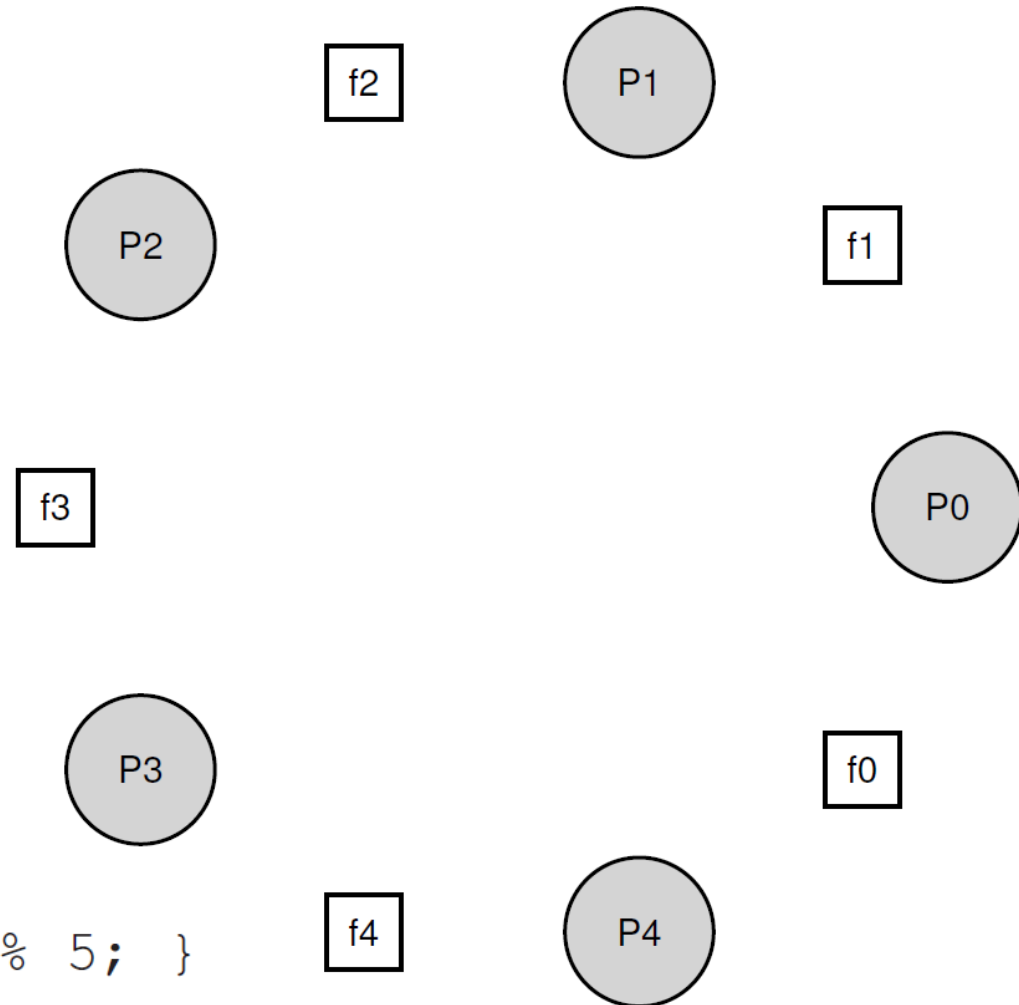
```
while (1) {  
    think();  
    get_forks(p);  
    eat();  
    put_forks(p);  
}
```

- A philosopher can only do one of the two things at a time

# Dining Philosophers Problem ... contd.

- Implement `get_forks()` and `put_forks()` such that:
  - There's no deadlock
  - No philosopher starves
  - As much concurrency as possible

```
int left(int p) { return p; }  
int right(int p) { return (p + 1) % 5; }
```



# Dining Philosophers - implementation

- Five semaphores (one for each fork) `sem_t forks[5].`
- Initialize each semaphore to 1

```
1 void get_forks(int p) {
2     sem_wait(&forks[left(p)]);
3     sem_wait(&forks[right(p)]);
4 }
5
6 void put_forks(int p) {
7     sem_post(&forks[left(p)]);
8     sem_post(&forks[right(p)]);
9 }
```

10 int main() {

# Dining Philosophers – breaking deadlock

- Break the dependency!
- Let at least one philosopher acquire forks in different order!

```
1 void get_forks(int p) {  
2     if (p == 4) {  
3         sem_wait(&forks[right(p)]);  
4         sem_wait(&forks[left(p)]);  
5     } else {  
6         sem_wait(&forks[left(p)]);  
7         sem_wait(&forks[right(p)]);  
8     }  
9 }
```

# Semaphores – an implementation

```
1  typedef struct __Zem_t {
2      int value;
3      pthread_cond_t cond;
4      pthread_mutex_t lock;
5  } Zem_t;
6
7  // only one thread can call this
8  void Zem_init(Zem_t *s, int value) {
9      s->value = value;
10     Cond_init(&s->cond);
11     Mutex_init(&s->lock);
12 }
13
14 void Zem_wait(Zem_t *s) {
15     Mutex_lock(&s->lock);
16     while (s->value <= 0)
17         Cond_wait(&s->cond, &s->lock);
18     s->value--;
19     Mutex_unlock(&s->lock);
20 }
21
22 void Zem_post(Zem_t *s) {
23     Mutex_lock(&s->lock);
24     s->value++;
25     Cond_signal(&s->cond);
26     Mutex_unlock(&s->lock);
27 }
```